

Лабораторна робота № 2

Тема. Практичне використання функцій, аргументів та декораторів.

Мета: навчитися створювати та використовувати функції, їх аргументи й декоратори.

Короткі теоретичні відомості

Функція – це іменована сутність, яка інкапсулює певну послідовність інструкцій (операторів) для виконання конкретного завдання. Розробник знає, що подати на вхід (аргументи) і що отримати на виході (результат), при цьому внутрішня реалізація залишається прихованою на рівні виклику. Функція створюється за допомогою ключового слова *def* після якого йде унікальне ім'я, дужки для параметрів і двокрапка. Тіло функції формується блоком коду з однаковим відступом (4 пробіли), що є обов'язковою синтаксичною вимогою мови. Приклад створення функції та її виклику:

```
def calculate_area(width, height):  
    return width * height
```

```
print(calculate_area(10, 20))
```

Параметр – це змінна, яка фігурує у сигнатурі функції.

Аргумент – це фактичне значення (дані), яке передається у функцію під час її виклику.

Параметри за обов'язковістю передавання поділяються на обов'язкові та необов'язкові.

Параметри за способом зіставлення з аргументом: позиційно-ключові, суто позиційні, суто ключові.

Параметри за варіативністю: **args* – збирає всі зайві позиційні аргументи у кортеж; ***kwargs* – збирає всі зайві ключові аргументи у словник.

Приклад:

```
def add(x, y): # обов'язкові  
    return x + y  
  
print(add(1, 2)) # позиційно  
print(add(x=1, y=2)) # за ключем
```

```

def add(x, y=2): # необов'язкові
    return x + y

print(add(2))

# சுто позиційні позначаються /.
# Усе, що ліворуч, передається лише за порядком.
def func(a, b, /, c):
    pass

func(1, 2, c=3)

# சுто ключові позначається як *.
# Усе, що праворуч, передається лише за іменем
def func(a, *, b, c):
    pass
func(1, b=2, c=3)

def sum_all(first, *rest): # *args
    return first + sum(rest)

print(sum_all(1, 2, 3, 3))

def print_data(**data): # **kwargs
    print(data)
print_data(name="Petro", age=18)

```

Декоратори – це інструмент, який дозволяє змінювати або розширювати поведінку функцій або класів без безпосередньої зміни їхнього вихідного коду. По суті, це реалізація шаблону проєктування «Декоратор» вбудована безпосередньо в синтаксис мови.

Приклад створення власного декоратора та його використання:

```

def Log(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print("Результат:", result)
        return result
    return wrapper

```

@Log

```
def add(a, b):  
    return a + b
```

add(5, 3)

Порядок виконання роботи

1. Уважно вивчити короткі теоретичні відомості та літературу.
2. Створити власний декоратор відповідно до варіанта та використати його.
3. Написати звіт з виконання лабораторної роботи.

Варіанти завдань

№ варіанта	Назва декоратора	Параметри (за замовчуванням)	Короткий опис функціоналу
1	@log	level="INFO"	Розширене логування викликів (час, аргументи, результат).
2	@timer	unit="sec"	Вимірює час виконання у різних одиницях.
3	@deprecated	message, alternative	Позначає функцію як застарілу з виводом попередження.
4	@uppercase	—	Перетворює рядковий результат функції на верхній регістр.
5	@double_return	—	Подвоює числовий результат, повернутий функцією (множить на 2).
6	@count_calls	—	Підраховує, скільки разів була викликана функція.
7	@slow_down	seconds = 1	Затримує виконання на задану кількість секунд.

Контрольні питання

1. Чим параметр відрізняється від аргументу?
2. Поясніть механізм передачі аргументів у Python?
3. Що таке **args* та ***kwargs*?
4. Що таке декоратор?
5. Що таке позиційно-ключові(/ та *) маркери в сигнатурі функції?